

Armazenando múltiplos atributos
em um único VBO

Introdução

Até agora vimos como armazenar atributos em VBOs separados, por exemplo:

- um VBO apenas para posições
- outro VBO apenas para normais

Outra abordagem muito comum é armazenar **todos os atributos de um vértice juntos**, no mesmo VBO. Isso é chamado de **layout intercalado (interleaved)**.

Nesse caso, cada vértice contém todos os seus dados consecutivamente na memória.

Exemplo

Exemplo de layout por vértice: x, y, z, nx, ny, nz

Ou seja:

- posição → 3 floats
- normal → 3 floats
- Total por vértice: 6 floats

Estrutura da memória no VBO:

v0.x	v0.y	v0.z	v0.nx	v0.ny	v0.nz
v1.x	v1.y	v1.z	v1.nx	v1.ny	v1.nz
v2.x	v2.y	v2.z	v2.nx	v2.ny	v2.nz
...					

Configurando os atributos no VAO

Agora precisamos dizer ao OpenGL como interpretar esses dados. Primeiro ativamos o VBO normalmente:

```
glBindBuffer(GL_ARRAY_BUFFER, vbo)
```

Configurando os atributos no VAO

Atributo 0 posição: a posição ocupa os 3 primeiros floats de cada vértice:

```
glVertexAttribPointer(  
    0,                # location no shader  
    3,                # número de componentes (x, y, z)  
    GL_FLOAT,  
    GL_FALSE,  
    6 * 4,            # stride (6 floats por vértice, 4 bytes cada)  
    ctypes.c_void_p(0) # offset  
)  
glEnableVertexAttribArray(0)
```

Configurando os atributos no VAO

Atributo 1 normal: A normal começa depois dos 3 floats da posição.

```
glVertexAttribPointer(  
    1,  
    3,  
    GL_FLOAT,  
    GL_FALSE,  
    6 * 4,  
    ctypes.c_void_p(3 * 4) # offset da normal  
)
```

```
glEnableVertexAttribArray(1)
```

O que é o **stride**

O stride indica a distância entre dois vértices consecutivos na memória.

Cada vértice possui:

- `x y z nx ny nz`

Ou seja:

- 6 floats

Como cada float tem 4 bytes:

- `stride = 6 * 4 = 24 bytes`

O que é o **offset**

O offset indica onde começa cada atributo dentro de um vértice.

Para posição:

- `offset = 0`

Para normal:

- `offset = 3 * 4 = 12 bytes`

Relação com o shader

Isso corresponde ao **layout** esperado no shader:

glsl:

- `layout(location = 0) in vec3 position;`
- `layout(location = 1) in vec3 normal;`